Private Exposure Registry :אפיון · Case 02

פרק 2 (אפיון) בתבנית תיק הפרויקט של מגשימים: פיצ'רים, טכנולוגיות, תרשים זרימה ומבנה בסיס נתונים. המסמך כתוב כך שמפתח אחר יוכל לבנות את המערכת ממנו. כל פיצ'ר מפורט ברמת "קוד במילים": מה הוא עושה, איך הוא עובד שלב אחר שלב, אילו פונקציות ומבני נתונים, אילו כלים מהתעשייה משלימים אותו, ומה הקלט והפלט שלו.

סקירת המערכת

מה המערכת עושה, במשפט אחד

Private Exposure Registry מאפשר לגופים פיננסיים מפוקחים לדווח את החשיפות שלהם לריבית ולמנוף בצורה מפוצלת ומוצפנת, כך ששלושה שרתים בשלוש רשויות פיקוח מחשבים יחד תמונה מצרפית של השוק, בלי שאף אחד מהם, ואף אחד אחר, יראה את הנתון של משתתף בודד.

מה השתנה מאז ה-POC ומודל האיום

- **ה-POC הוכיח את הליבה:** פיצול לחלקים, צבירה ופתיחה של הסכום בלבד, ב-0.11 שניות ל-500 משתתפים. אבל הוא הניח שרתים "ישרים אך סקרנים" ומשתתפים ישרים.
- **מודל האיום מצא שש פרצות, וזה המסמך שבו המודל משנה את התכנון יותר מבכל השאר:** התקפת סיביל (קרנות מזויפות שעוקפות את הסף), שרת שמשבש חלקים, משתתף שמכחיש, התקפת הפרשים, שרת שנופל, ומשתתף שמשקר.
- **בדקנו מה התעשייה כבר בנתה:** יש סטנדרט שעושה בדיוק את הליבה הזו. Prio3, חלק מהטיוטה VDAF (Verifiable Distributed Aggregation Functions) של קבוצת המחקר הקריפטוגרפית של IETF, שהגיעה לגרסה 21 באוגוסט 2026. Prio שימש בפועל לסטטיסטיקות פרטיות של אפליקציות התראות הקורונה. לכן לא נבנה פרוטוקול משלנו: נבנה על הסטנדרט, ונוסיף מעליו את מה שהוא לא נותן.

החלטת תכנון מרכזית: כמה שרתים צריכים להיות ישרים?

מודל האיום השאיר החלטה פתוחה: כל שלושת השרתים נחוצים (סודי יותר, אבל שרת אחד שנופל עוצר הכול), או כל שניים מספיקים (זמין יותר, אבל שניים בקנוניה שוברים סודיות). ההחלטה: כל שלושת החלקים נחוצים. הסיבה: ב-Prio3, סודיות נשמרת כל עוד לפחות שרת אחד מתוך כולם ישר. כדי לחשוף נתון של קרן, כל שלוש הרשויות צריכות לשתף פעולה בסתר, וזה תרחיש רחוק בהרבה מקנוניה של שתיים. את מחיר הזמינות משלמים בתפעול: כל שרת הוא אשכול עם גיבוי, ודיווחים מוצפנים נשמרים עד שהשרת חוזר (פיצ'ר 8). זו פשרה מודעת: מעדיפים עיכוב בדוח על פני דליפה.

רשימת הפיצ'רים

- פיצ'ר 1, רישום משתתפים וזהות (Participant Onboarding & Credentials)
- פיצ'ר 2, תבנית דיווח וקידוד מקומי (Reporting Template & Local Encoding)
- פיצ'ר 3, התחייבות קריפטוגרפית וקבלות (Commitments & Receipts)
- פיצ'ר 4, פיצול ואימות תקינות (Prio3 Sharding & Validity Proofs)
- פיצ'ר 5, צבירה בין שלושת השרתים (Three-Aggregator Protocol)
- פיצ'ר 6, אימות נכונות התוצאה (Verifiable Aggregate)
- פיצ'ר 7, קטלוג שאלות, שער סף ותקציב פרטיות (Query Catalog, Threshold & Privacy Budget)
- פיצ'ר 8, תקשורת, הצפנה ותפעול (Transport, HPKE & Operations)
- פיצ'ר 9, יומן ביקורת ושקיפות (Audit & Transparency Log)
- פיצ'ר 10, דוחות לרגולטור ופרסום לציבור (Regulator Reports & Public Release)

פיצ'ר 1: רישום משתתפים וזהות (Participant Onboarding & Credentials)

מה הפיצ'ר עושה?

קובע מי רשאי לדווח. רק גוף מפוקח אמיתי, שזוהה על ידי רגולטור, מקבל אישור השתתפות. זה המענה להתקפת הסיביל ממודל האיום: אם אי אפשר לרשום קרן מזויפת, אי אפשר "לנפח" תא כדי לעבור את סף המינימום.

איך הפיצ'ר עובד?

זיהוי לפי LEI

כל גוף פיננסי בעולם שפועל בשווקים מחזיק LEI (Legal Entity Identifier): מזהה בן 20 תווים לפי תקן ISO 17442, שמונפק ומאומת על ידי ארגון GLEIF ושותפיו. הרגולטור בודק שה-LEI של המבקש פעיל במאגר הציבורי של GLEIF, ושהוא מופיע ברשימת הגופים שבפיקוחו. גוף אחד, LEI אחד: אי אפשר לרשום עשר קרנות מאותו גוף.

אישור השתתפות אנונימי

כאן יש בעיה עדינה: אם השרתים יודעים איזה LEI שלח כל דיווח, הם יודעים מי משתתף, ומודל האיום סימן את עצם ההשתתפות כמידע רגיש. הפתרון: אסימונים עיוורים (blind tokens). הרגולטור מנפיק לכל משתתף מאומת, פעם בתקופת דיווח, אסימון חתום אחד, בשיטת חתימה עיוורת: הרגולטור חותם בלי לראות את האסימון עצמו. כשהמשתתף שולח דיווח, הוא מצרף את האסימון. השרתים בודקים שהחתימה של הרגולטור תקינה ושהאסימון לא שומש, אבל לא יכולים לקשר אותו ל-LEI. כך: רק גופים אמיתיים מדווחים, כל גוף מדווח פעם אחת, ואף שרת לא יודע מי.

ספירת משתתפים אמיתיים

סף המינימום בפיצ'ר 7 נספר לפי מספר האסימונים התקפים בתא, לא לפי מספר הדיווחים. מכיוון שכל גוף מקבל אסימון אחד, "חמישה משתתפים" הם באמת חמישה גופים שונים.

טכנולוגיות ותשתיות תעשייה משלימות

- **GLEIF API:** הממשק הציבורי של מאגר ה-LEI העולמי. התרומה: אימות זהות לפי מקור עולמי מוסמך, ולא לפי רשימה פנימית.
- **Privacy Pass (RFC 9576 ו-RFC 9578):** סטנדרט של IETF לאסימונים עיוורים, שמשמש בפועל את Cloudflare ו-Apple כדי להוכיח "אני משתמש אמיתי" בלי לחשוף "איזה משתמש". התרומה: המנגנון שתיארנו הוא בדיוק Privacy Pass, עם RSA Blind Signatures (RFC 9474). לא ממציאים קריפטוגרפיה.
- **smallstep step-ca:** רשות אישורים פתוחה לניהול תעודות X.509. התרומה: הרגולטורים עצמם, והשרתים, מזוהים זה מול זה בתעודות.

מבנה נתונים

```
Participant {            # stored at the regulator only
  lei: str               # ISO 17442, 20 chars
  regulator_id: str
  sector: str
  verified_at: datetime
  status: Literal["active", "suspended"]
}

ParticipationToken {     # what the aggregators see
  period_id: str         # e.g. "2026-Q4"
  token: bytes           # unblinded token
```

```
  signature: bytes       # regulator RSA blind signature
}
```

קלט ופלט

קלט: בקשת הצטרפות עם LEI. פלט: רשומת Participant אצל הרגולטור, ואסימון השתתפות אנונימי אחד לכל תקופה.

ספריות ופונקציות מרכזיות

שימוש	API / ספרייה	פעולה
GET /api/v1/lei-records/{lei}	GLEIF API	אימות LEI
()blind(), blind_sign(), finalize	Privacy Pass, RFC 9474	חתימה עיוורת
X.509, mTLS	step-ca	תעודות שרתים

פיצ'ר 2: תבנית דיווח וקידוד מקומי (Local & Reporting Template (Encoding

מה הפיצ'ר עושה?

אפליקציית לקוח שרצה אצל המשתתף, מקבלת את ספר ההלוואות שלו, ומחשבת ממנו את וקטור הדיווח האחיד. הנתונים הגולמיים לא עוזבים את המחשב של המשתתף. זו בדיוק החלטת התכנון שה-POC אישר: הקיבוץ למדרגות נעשה מקומית, כך שהשרתים צריכים רק לחבר.

איך הפיצ'ר עובד?

מבנה הווקטור

הווקטור מורכב מתאים קבועים, שמוגדרים בגרסת התבנית של התקופה:

- **חשיפה לפי מדרגת ריבית וענף:** ארבע מדרגות ריבית (מתחת ל-3%, 3 עד 4, 4 עד 5, מעל 5), כפול שבעה ענפים. כל תא הוא סך החשיפה בדולרים, מעוגל למיליון.
- **מדרגת מנוף:** חמש מדרגות (מתחת ל-2, 2 עד 3, 3 עד 4, 4 עד 5, מעל 5). וקטור "one-hot": אחד במדרגה של המשתתף, אפס בשאר.
- **חשיפה לריבית משתנה מול קבועה:** שני תאים.
- **תא ספירה:** תמיד 1, כדי לספור משתתפים באותה צבירה.

חיתוך (clipping)

כל תא חשיפה נחתך לתקרה (למשל 400 מיליון לתא). הסיבה כפולה: היא מגבילה כמה משתתף בודד יכול להשפיע על הסכום (נדרש לחישוב הרעש בפיצ'ר 7), והיא מגבילה את הנזק שמשתתף שמנסה לשבש יכול לעשות. המשתתף מקבל אזהרה כשתא נחתך, וכמה מהערך שלו נחתך נשמר מקומית לביקורת.

קידוד לשדה סופי

הקריפטוגרפיה עובדת על מספרים שלמים בשדה סופי (מודולו מספר ראשוני). כל ערך מומר למספר שלם לא שלילי, וכל ערך מוגבל למספר ביטים שהתבנית קובעת (למשל 9 ביטים לתא חשיפה, כי 400 קטן מ-512). הגבלת הביטים היא מה שמאפשר לבדוק בהמשך שהערך בטווח (פיצ'ר 4).

מה רואה המשתתף

לפני שליחה, האפליקציה מציגה למשתתף את הווקטור שלו בטבלה קריאה ("את זה אתם עומדים לדווח"), והמשתתף מאשר. מה שלא מופיע בטבלה לא נשלח.

טכנולוגיות ותשתיות תעשייה משלימות

- **Rust עם WebAssembly:** אפליקציית הלקוח כתובה ב-Rust ומהודרת גם לאפליקציה שולחנית וגם ל-WebAssembly בדפדפן. התרומה: אותה ספריית קריפטוגרפיה (פיצ'ר 4) בכל מקום, וזיכרון בטוח.
- **Apache Arrow:** פורמט עמודתי בזיכרון. התרומה: קריאת ספר הלוואות של מאות אלפי שורות מקובצי Parquet או CSV של המשתתף, וקיבוץ מהיר.
- **JSON Schema לתבנית:** כל גרסת תבנית מתפרסמת כסכמה. התרומה: הלקוח, השרתים והרגולטור מסכימים בדיוק על מבנה הווקטור, וכל שינוי בתבנית הוא גרסה חדשה עם מזהה.

מבנה נתונים

```
Template {
  template_id: str            # "exp-v3"
  rate_buckets: list[float]   # [3.0, 4.0, 5.0]
  sectors: list[str]          # 7 sectors
  lev_buckets: list[float]    # [2, 3, 4, 5]
  clip_musd: int              # 400
  bits_per_cell: int          # 9
}

LocalReport {                 # never leaves the participant's machine in clear
  template_id: str
  cells: list[int]            # length L, each in [0, 2^bits)
  clipped_cells: list[int]    # indices that were clipped (kept locally)
}
```

קלט ופלט

קלט: ספר ההלוואות של המשתתף (קובץ מקומי), ותבנית התקופה. פלט: וקטור מקודד באורך קבוע, שמוכן לשלב ההתחייבות והפיצול.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
ParquetRecordBatchReader	Apache Arrow (arrow-rs)	קריאת ספר הלוואות
HashMap<(rate_bucket, sector), <u64	Rust std	קיבוץ
ייצוא פונקציות ל-JavaScript	wasm-bindgen	ריצה בדפדפן

פיצ'ר 3: התחייבות קריפטוגרפית וקבלות (Commitments & Receipts)

מה הפיצ'ר עושה?

לפני שהמשתתף מפצל את הנתונים, הוא "נועל" אותם בהתחייבות קריפטוגרפית: מספר שמוכיח מה הוא דיווח, בלי לחשוף את זה. מענה לאיום "משתתף שמכחיש": מאוחר יותר, בביקורת, אפשר לבקש מהמשתתף לפתוח את ההתחייבות, והוא לא יכול לטעון שדיווח משהו אחר. ובונוס חשוב: ההתחייבויות מאפשרות לבדוק שהשרתים חישבו נכון (פיצ'ר 6).

איך הפיצ'ר עובד?

התחייבות Pedersen

לכל תא בווקטור, המשתתף מחשב התחייבות Pedersen: נקודה על עקום אליפטי, שהיא x כפול נקודה G ועוד r כפול נקודה H, כש-x הוא ערך התא ו-r מספר אקראי סודי. שלוש תכונות הופכות אותה למתאימה:

- **מסתירה:** בלי r, ההתחייבות לא מגלה כלום על x.
- **מחייבת:** אי אפשר למצוא x אחר ו-r אחר שנותנים אותה התחייבות.
- **והתכונה החשובה ביותר כאן, הומומורפית:** חיבור של התחייבויות שווה להתחייבות על הסכום. אם מחברים את ההתחייבויות של כל המשתתפים, מקבלים התחייבות לסכום הכולל, בלי לפתוח אף אחת מהן.

מה נשלח

ההתחייבויות (וקטור של נקודות על העקום) נשלחות לרגולטור עם הדיווח, ונשמרות ביומן הביקורת (פיצ'ר 9). את r המשתתף לא שומר אצלו בלבד: הוא מפצל גם אותו, כמו את הנתונים, ושולח את החלקים לשלושת השרתים יחד עם חלקי הנתונים. כך השרתים יוכלו לצבור גם את r, וזה מה שמאפשר את בדיקת הנכונות בפיצ'ר 6.

קבלה חתומה

השרת המוביל מחזיר למשתתף קבלה חתומה: מזהה הדיווח, טביעת האצבע של ההתחייבויות, וזמן הקבלה. אם הדיווח לא נכלל בצבירה, למשתתף יש הוכחה שהוא דיווח בזמן.

טכנולוגיות ותשתיות תעשייה משלימות

- **Ristretto255 (RFC 9496):** קבוצה מתמטית מעל Curve25519, שמתוכננת בדיוק לפרוטוקולים כאלה ומונעת מלכודות ידועות של העקום הגולמי. התרומה: התחייבויות קצרות (32 בייט לתא) ופעולות מהירות.
- **curve25519-dalek (Rust):** ספריית המימוש המוכרת של Ristretto255, שמשמשת גם במימושי Bulletproofs ובפרוטוקולים של Signal. התרומה: מימוש מבוקר ויעיל, בלי לגעת במתמטיקה של העקום ידנית.
- **Hash-to-curve (RFC 9380):** יצירת הנקודה H מתוך מחרוזת ציבורית, כך שאף אחד לא יודע את היחס בינה לבין G. התרומה: זו התכונה שמבטיחה שההתחייבות מחייבת. בלי זה, מי שבחר את H יכול היה לרמות.

מבנה נתונים

```
CommitmentSet {
  report_id: str          # random 16 bytes, hex
  template_id: str
  commitments: list[bytes]   # L x 32 bytes, Ristretto255 points
}

Receipt {
  report_id: str
  commitments_sha256: str
  received_at: datetime
  leader_signature: bytes    # Ed25519
}
```

קלט ופלט

קלט: וקטור מקודד מפיצ'ר 2. פלט: התחייבויות לכל תא, ערכי r (שמפוצלים יחד עם הנתונים), וקבלה חתומה.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
התחייבות	curve25519-dalek	RistrettoPoint, Scalar, multiscalar_mul
נקודה שנייה	RFC 9380	hash_to_ristretto255("exposure-registry/H")
קבלה	ed25519-dalek	()SigningKey::sign

פיצ'ר 4: פיצול ואימות תקינות (Prio3 Sharding & Validity Proofs)

מה הפיצ'ר עושה?

מפצל את הווקטור לשלושה חלקים, ומצרף הוכחה שהנתונים תקינים, בלי לחשוף אותם. "תקינים" כאן פירושו: כל תא בטווח המותר, ווקטור המנוף באמת one-hot (בדיוק אחד מחמשת התאים הוא 1). מענה חלקי לאיום "משתתף שמשקר": משתתף לא יכול לשלוח ערך שלילי שיקזז את האחרים, או מיליארד בתא אחד.

איך הפיצ'ר עובד?

Prio3Histogram-ו Prio3SumVec

הטיוטה VDAF מגדירה כמה גרסאות של Prio3. אנחנו משתמשים בשתיים: Prio3SumVec לתאי החשיפה (וקטור של סכומים, כל אחד מוגבל למספר ביטים), ו-Prio3Histogram למדרגת המנוף: המשתתף מוסר רק את מספר המדרגה, וההוכחה מבטיחה שהוא תורם בדיוק 1 למדרגה אחת. כל אחת מהן מוגדרת לפי אורך הווקטור ומספר הביטים לתא.

איך פיצול עם הוכחה עובד

הלקוח מפרק כל ערך לביטים, ומפצל את הביטים לשלושה חלקים אקראיים (כמו ב-POC, אבל בשדה של 128 ביט). לצד זה הוא מחשב הוכחה קצרה מסוג FLP (Fully Linear Proof), שאומרת בעצם "כל ביט הוא 0 או 1, וכל ערך הוא הסכום של הביטים שלו". גם ההוכחה מפוצלת בין השרתים. כל שרת בודק את החלק שלו, השרתים מחליפים ביניהם תוצאה קצרה של הבדיקה, ואם היא נכשלת, הדיווח נזרק. אף שרת לא לומד שום דבר על הערכים עצמם.

מה ההוכחה לא מוכיחה

ההוכחה מבטיחה שהערך בטווח, לא שהוא נכון. משתתף שמדווח 50 מיליון כשהאמת היא 300 מיליון עובר את הבדיקה. זה האיום שמודל האיום סימן כגדול ביותר, ואין לו פתרון קריפטוגרפי מלא. המענה החלקי שלנו נמצא בפיצ'ר 9: ביקורת מדגמית שבה משתתף נדרש לפתוח את ההתחייבות שלו מול הדיווח הרגולטורי הרגיל שלו.

טכנולוגיות ותשתיות תעשייה משלימות

- **libprio-rs:** מימוש ה-Rust של VDAF ו-Prio3, שמתוחזק על ידי ISRG (הארגון שמפעיל גם את Let's Encrypt) ומשמש בשירות Divvi Up שלהם. התרומה: ממשתמשים בקוד שכבר רץ בייצור, ולא כותבים FLP בעצמנו.
- **VDAF (draft-irtf-cfrg-vdaf):** הטיוטה של קבוצת המחקר הקריפטוגרפית של IETF. התרומה: מפרט פומבי, עם הוכחות אבטחה מתועדות, שקריפטוגרפים מבחוץ יכולים לבדוק.
- **Bulletproofs (כחלופה שנשקלה):** הוכחות טווח קצרות שמשמשות במטבע Monero. נשקלו ונדחו: הן מוכיחות טווח על התחייבות בודדת, אבל לא משתלבות בפיצול בין שרתים כמו FLP, והאימות שלהן כבד יותר.

מבנה נתונים

```
Report {
  report_id: bytes[16]
  task_id: bytes[32]          # identifies template + period + VDAF params
  public_share: bytes
  input_shares: list[bytes]   # 3 shares, each later HPKE-encrypted (feature 8)
  commitment_randomness_shares: list[bytes]   # additive shares of r over the Ristretto255
scalar field
  token: ParticipationToken
}
VdafConfig { kind: Literal["Prio3SumVec", "Prio3Histogram"]; length: int; bits: int;
num_aggregators: 3 }
```

קלט ופלט

קלט: וקטור מקודד וערכי r. פלט: Report עם שלושה חלקים והוכחת תקינות מפוצלת.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
Prio3::new_sum_vec(3, bits, len, ()chunk), shard	libprio-rs	פיצול והוכחה
Prio3::new_histogram(3, 5, chunk)	libprio-rs	מדרגת מנוף
CSPRNG של מערכת ההפעלה	getrandom	אקראיות

פיצ'ר 5: צבירה בין שלושת השרתים (Three-Aggregator Protocol)

מה הפיצ'ר עושה?

שלושה שרתים, אחד אצל כל רשות פיקוח, מקבלים את החלקים, בודקים יחד את תקינות כל דיווח, וצוברים את החלקים התקינים. בסוף תקופת הדיווח, כל שרת מחזיק סכום חלקי אחד, שנראה כמו רעש.

איך הפיצ'ר עובד?

תפקידים

השרת המוביל (Leader) מקבל את כל הדיווחים מהמשתתפים, ומעביר לשני השרתים העוזרים (Helpers) את החלקים שלהם, מוצפנים כך שרק הם יכולים לפתוח (פיצ'ר 8). זה המבנה של DAP (Distributed Aggregation Protocol), הפרוטוקול של קבוצת PPM ב-IETF שבנוי על VDAF. DAP מוגדר לשני שרתים. אנחנו מרחיבים לשלושה, לפי טופולוגיית ה-star שהטיוטה VDAF מתארת למספר שרתים כלשהו.

שלבי עבודה

- **קליטה:** המוביל בודק שהאסימון חתום על ידי רגולטור ושלא שומש (רשימת אסימונים שנוצלו, לכל תקופה), ושה-task_id תקף. דיווחים כפולים (אותו report_id) נזרקים.
- **הכנה (prepare):** כל שרת פותח את החלק שלו ומריץ את שלב האימות של Prio3. השרתים מחליפים הודעות אימות קצרות דרך המוביל. דיווח שנכשל מסומן כנזרק, עם סיבה.
- **צבירה:** כל שרת מוסיף את החלק של כל דיווח תקין לסכום החלקי שלו, וגם את החלק של ערכי r (פיצ'ר 3). הצבירה נעשית לפי "אצוות" (batch): קבוצת דיווחים של אותה תקופה.
- **שמירה:** החלקים של כל דיווח נשמרים עד סוף תקופת השימור (למשל 90 יום), כדי לאפשר חיפוש בינארי אם תתגלה בעיה (פיצ'ר 6). אחרי התקופה הם נמחקים.

הגנה מפני שימוש חוזר

כל שרת שומר אילו report_id נכללו באילו אצוות. דיווח לא יכול להיכלל בשתי אצוות, וזה מונע ממי שקיבל גישה לשרת לבנות "אצווה" מלאכותית שמכילה דיווח אחד.

טכנולוגיות ותשתיות תעשייה משלימות

- **Janus:** המימוש של ISRG לשרת DAP, ב-Rust, שמריץ את Divvi Up בייצור. התרומה: בסיס לשרתים שלנו. נדרשת התאמה לשלושה שרתים ולתוספות שלנו (ההתחייבויות).
- **PostgreSQL:** בסיס הנתונים של כל שרת, עם רמת בידוד SERIALIZABLE לפעולות הצבירה. התרומה: גם אם שני תהליכים מנסים לצבור את אותו דיווח במקביל, רק אחד יצליח.
- **Kubernetes:** כל שרת רץ כאשכול, עם עותקים מרובים ושחזור אוטומטי. התרומה: המענה לזמינות שבחרנו בהחלטת התכנון.

מבנה נתונים

```
AggregationJob { job_id; task_id; batch_id; report_ids: list[bytes]; state:
Literal["prep", "done", "failed"] }
AggregateShare {
  task_id: bytes
  batch_id: str
  aggregator_index: int          # 0 leader, 1-2 helpers
  share: bytes                   # field-element vector
  r_share: bytes                 # sum of commitment-randomness shares
  report_count: int
  checksum: bytes[32]            # XOR of report_id hashes, for cross-check
}
```

קלט ופלט

קלט: דיווחים מהמשתתפים. פלט: AggregateShare אחד בכל שרת לכל אצווה, ורשימת דיווחים שנזרקו עם הסיבה.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
שרת DAP	Janus	aggregator, aggregation job driver
אימות	libprio-rs	prepare_init(), prepare_next(), ()aggregate
מסד נתונים	PostgreSQL, sqlx	transactions SERIALIZABLE

פיצ'ר 6: אימות נכונות התוצאה (Verifiable Aggregate)

מה הפיצ'ר עושה?

Prio3 מבטיח סודיות כל עוד שרת אחד ישר, אבל, כפי שהטיוטה עצמה כותבת, הרגולטור שמקבל את התוצאה צריך לסמוך על כך שהשרתים הריצו את הפרוטוקול נכון. מודל האיום סימן את זה כאיום "שרת שמשבש חלקים". הפיצ'ר הזה הופך את התוצאה לניתנת לבדיקה: הרגולטור לא צריך לסמוך, הוא יכול לוודא.

איך הפיצ'ר עובד?

הבדיקה ההומומורפית

זוכרים את התכונה ההומומורפית של התחייבות Pedersen מפיצ'ר 3? הרגולטור מחזיק את ההתחייבויות של כל הדיווחים באצווה (מיומן הביקורת). הוא מחבר אותן ומקבל התחייבות לסכום הכולל. כשהשרתים מגישים את הסכומים החלקיים שלהם, הרגולטור מחבר אותם ומקבל את הסכום X, וגם את סכום ערכי ה-r שלהם, R. עכשיו הוא בודק: האם X כפול G ועוד R כפול H שווה לסכום ההתחייבויות? אם כן, הסכום נכון. אם שרת אחד שינה את החלק שלו, השוויון נשבר.

מציאת הדיווח הבעייתי

אם הבדיקה נכשלת, ייתכן ששרת שיבש, וייתכן שמשתתף שלח התחייבות שלא תואמת לנתונים שפיצל. כדי להבחין, הרגולטור מבקש מהשרתים לצבור מחדש חצי מהאצווה, ובודק אותו מול ההתחייבויות של אותו חצי. חיפוש בינארי כזה מוצא דיווח בודד שגוי בכ-log של מספר הדיווחים צעדים (9 צעדים ל-500 דיווחים). זו הסיבה לשמירת החלקים בפיצ'ר 5.

מגבלה חשובה

הבדיקה נעשית על הסכום לפני הוספת רעש הפרטיות (פיצ'ר 7). לכן היא זמינה רק לרגולטורים, שמותר להם לראות את הסכום המדויק. הפרסום לציבור, עם רעש, אינו ניתן לבדיקה באותה דרך, ונשען על כך שהרגולטור כבר בדק את הסכום שממנו הוא נגזר.

טכנולוגיות ותשתיות תעשייה משלימות

- **curve25519-dalek multiscalar_mul:** חישוב סכום של מאות נקודות על העקום בפעולה אחת, באלגוריתם Pippenger. התרומה: בדיקת אצווה של אלפי דיווחים בשברירי שנייה.
- **עקרון "Trust but Verify" מעולם ה-Certificate Transparency:** כמו שדפדפנים לא סומכים על רשות אישורים אלא בודקים ביומן ציבורי, הרגולטור לא סומך על השרתים אלא בודק מול התחייבויות. התרומה: מודל אמון מוכר, לא חדש.

מבנה נתונים

```
VerificationResult {
  batch_id: str
  n_reports: int
  commitments_sum: bytes        # Ristretto point per cell
  opened_sum: list[int]
  opened_r: list[int]
  ok: bool
  bisection_trace: list[tuple[int, int, bool]] | None   # (start, end, ok)
  offending_report_id: str | None
}
```

קלט ופלט

קלט: שלושת ה-AggregateShare, וההתחייבויות של כל הדיווחים באצווה. פלט: סכום מאומת, או זיהוי הדיווח או השרת שגרמו לכשל.

ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
סכום התחייבויות	curve25519-dalek	RistrettoPoint::multiscalar_mul / ()sum
חיפוש בינארי	Rust std	רקורסיה על טווחי אינדקסים

פיצ'ר 7: קטלוג שאלות, שער סף ותקציב פרטיות (Query Catalog, Threshold & Privacy Budget)

מה הפיצ'ר עושה?

קובע אילו שאלות מותר לשאול, איזה תא מותר לפתוח, וכמה רעש להוסיף לפרסום. מענה לשני איומים ממודל האיום: התקפת הפרשים (שאלות שההבדל ביניהן חושף משתתף) ודליפה דרך תוצאה בתא קטן.

איך הפיצ'ר עובד?

קטלוג סגור

אין שאלות חופשיות. הקטלוג הוא רשימה של שאלות ליניאריות קבועות על תאי הווקטור, למשל "סך החשיפה בריבית מעל 5%" (סכום של שבעה תאים), "חלק הענף הגדול ביותר", או "התפלגות המנוף". כל שאלה מוגדרת כווקטור משקלות על התאים. הוספת שאלה לקטלוג מחייבת אישור של שלושת הרגולטורים, וגרסה חדשה של הקטלוג.

בדיקת הפרשים אוטומטית

לפני שגרסה של הקטלוג מאושרת, תוכנה בודקת כל זוג שאלות: אם ההפרש ביניהן הוא תא בודד (או קבוצת תאים) שבו עלולים להיות פחות מ-K משתתפים, הזוג נדחה. זה מונע את התרחיש ממודל האיום ("כל הענפים" מול "כל הענפים חוץ מאנרגיה") כבר בשלב התכנון.

שער הסף

כל תא נפתח לרגולטור רק אם מספר האסימונים התקפים שתרמו לו (פיצ'ר 1) גדול או שווה ל-K=5. תא חסום מוצג כ"פחות מ-5 משתתפים". שאלה שתלויה בתא חסום מחזירה תשובה רק אם השאלה כוללת מספיק תאים פתוחים, ואחרת נחסמת.

רעש ותקציב לפרסום הציבורי

לרגולטורים: הסכום המדויק, מאומת (פיצ'ר 6). לציבור: הסכום עם רעש Discrete Gaussian, שמכויל לרגישות (תקרת החיתוך מפיצ'ר 2) ולתקציב הפרטיות. התקציב נמדד ב- zCDP (zero-Concentrated Differential Privacy), שבו תקציבים של כמה פרסומים מצטברים בחיבור פשוט. לכל תקופה יש תקציב כולל, כל פרסום "מוציא" ממנו, וכשהוא נגמר, אין עוד פרסומים לאותה תקופה. ה-POC הראה שבאפסילון 1, השאלה "כמה מהחשיפה מעל 5%" יצאה 17.5% מול 16.8% אמיתי: שימושי גם בפרטיות חזקה.

טכנולוגיות ותשתיות תעשייה משלימות

- **OpenDP:** ספרייה פתוחה מבית Harvard ו-Microsoft לפרטיות דיפרנציאלית, עם מנגנון Discrete Gaussian ומעקב תקציב מובנה. התרומה: רעש מכויל וספירת תקציב לפי שיטה מוכחת, בלי לממש בעצמנו.
- **zCDP (Steinke, 2016 ו-Bun):** הגדרת פרטיות שבה הרכבת פרסומים פשוטה ומדויקת יותר מאשר בהגדרה הקלאסית. התרומה: יותר פרסומים שימושיים לאותו תקציב.
- **הלשכה למפקד של ארה"ב:** השתמשה בפרטיות דיפרנציאלית בפרסום מפקד 2020. התרומה: תקדים לשימוש ממשלתי ברעש מכויל בנתונים רשמיים.

מבנה נתונים

```
CatalogQuery { query_id: str; version: int; title: str; weights: dict[int, float]; public:
bool; rho_cost: float }
Catalog { version: int; queries: list[CatalogQuery]; approved_by: list[str];
differencing_check_passed: bool }
```

```
BudgetLedger { period_id: str; rho_total: float; rho_spent: float; entries:
list[tuple[str, float, datetime]] }
```

קלט ופלט

קלט: סכום מאומת, ספירת משתתפים לכל תא, וקטלוג. פלט: תשובות לרגולטור (מדויקות, עם תאים חסומים מסומנים), ותשובות לציבור (עם רעש), ועדכון התקציב.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
make_gaussian(..., mechanism discrete)	OpenDP	רעש
zCDP accounting, make_zCDP_to_approxDP	OpenDP	תקציב
כל זוג שאלות, symmetric difference	Python	בדיקת הפרשים

פיצ'ר 8: תקשורת, הצפנה ותפעול (Transport, HPKE & Operations)

מה הפיצ'ר עושה?

מבטיח שכל חלק מגיע רק לשרת שלו, שאף אחד בדרך לא יכול לקרוא או לשנות אותו, ושהמערכת ממשיכה לעבוד כששרת אחד לא זמין. מענה לאיום "שרת אחד נופל" במסגרת ההחלטה שקיבלנו (כל שלושת החלקים נחוצים).

איך הפיצ'ר עובד?

הצפנה לכל שרת

הלקוח מצפין כל חלק במפתח הציבורי של השרת שאליו הוא מיועד, בשיטת HPKE (Hybrid Public Key Encryption, RFC 9180), כמו ב-DAP. המשמעות: גם השרת המוביל, שמעביר את החלקים של העוזרים, לא יכול לקרוא אותם. הוא מחזיק בפועל שלושה מעטפות סגורות, ויכול לפתוח רק את שלו. מפתחות ה-HPKE של כל שרת מתפרסמים בכתובת קבועה, עם מזהה מפתח, ומתחלפים כל רבעון.

ערוצים בין השרתים

השרתים מזהים זה את זה בתעודות X.509 בערוץ mTLS. כל בקשה בין שרתים חתומה גם ברמת ההודעה, כך שיומן הביקורת יכול להוכיח מי שלח מה.

זמינות

- דיווח שמגיע כששרת עוזר לא זמין נשמר אצל המוביל כשהוא מוצפן (המוביל לא יכול לפתוח את חלק העוזר בכל מקרה), ומועבר כשהעוזר חוזר.
- אצווה נסגרת רק כשכל שלושת השרתים סיימו. דוח שמתעכב מסומן ככזה, עם סיבה.
- כל שרת רץ בשלושה עותקים לפחות, עם מסד נתונים משוכפל.
- **היעד:** זמינות חודשית של 99.9% לכל שרת. גם כך, בהסתברות קטנה, דוח יתעכב בדיוק ברגע של לחץ. זו הפשרה שבחרנו, וכתבנו אותה.

מפתחות במכשיר מאובטח

המפתח הפרטי של HPKE של כל שרת, ומפתח החתימה של הרגולטור לאסימונים, נשמרים במודול חומרה מאובטח (HSM) או בשירות ענן מקביל, ולא יוצאים ממנו.

טכנולוגיות ותשתיות תעשייה משלימות

- **HPKE (RFC 9180) עם המימוש hpke-rs:** התקן שמשמש גם ב-TLS Encrypted Client Hello וב-MLS. התרומה: הצפנה מודרנית ומוכחת בתבנית אחת.
- **mTLS עם Envoy:** זיהוי הדדי של השרתים, כמו בכל מערכת מיקרו-שירותים רצינית.
- **HSM או Cloud KMS:** מפתחות שלא ניתנים לגניבה גם בפריצה מלאה לשרת.
- **Prometheus ו-Grafana:** ניטור זמינות, קצב דיווחים, ושיעור דיווחים שנזרקו בכל שרת.

מבנה נתונים

```
HpkeConfig { config_id: int; kem: "X25519-HKDF-SHA256"; kdf: "HKDF-SHA256"; aead: "AES-128-GCM"; public_key: bytes; valid_until: date }
EncryptedShare { aggregator_index: int; config_id: int; enc: bytes; ciphertext: bytes }
PendingForward { report_id; helper_index: int; encrypted_share: EncryptedShare; queued_at: datetime }
```

קלט ופלט

קלט: חלקים מפוצלים מפיצ'ר 4. פלט: מעטפות מוצפנות לכל שרת, ערוצים מאומתים, ותור העברה עמיד לנפילות.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
()setup_sender(), seal	hpke-rs	הצפנה
mTLS	rustls, Envoy	ערוצים
counters, histograms	Prometheus	ניטור

פיצ'ר 9: יומן ביקורת ושקיפות (Audit & Transparency Log)

מה הפיצ'ר עושה?

רושם כל פעולה משמעותית ביומן שאי אפשר לשנות או למחוק ממנו: אילו אצוות נצברו, אילו שאלות נשאלו, כמה תקציב פרטיות נוצל, אילו גרסאות קטלוג אושרו. ומאפשר ביקורת מדגמית על משתתפים, שהיא המענה החלקי היחיד לאיום "משתתף שמשקר".

איך הפיצ'ר עובד?

יומן מבוסס עץ Merkle

כל רשומה מתווספת כעלה בעץ Merkle. שורש העץ מתפרסם מדי יום. כל מי שמחזיק שורש ישן יכול לקבל "הוכחת עקביות" שהיומן החדש הוא המשך שלו, בלי שנמחק או שונה דבר. זה בדיוק המנגנון של Certificate Transparency, שבזכותו אי אפשר להנפיק תעודת אבטחה לאתר בסתר.

מה נכנס ליומן

- התחייבויות של כל דיווח (פיצ'ר 3), לפי report_id, בלי זהות.
- **כל אצווה:** מזהה, מספר דיווחים, מספר שנזרקו, תוצאת האימות (פיצ'ר 6).
- **כל שאלה ופרסום:** מי שאל, איזו שאלה מהקטלוג, כמה תקציב נוצל.
- כל גרסת קטלוג ותבנית, עם האישורים.

ביקורת מדגמית על משתתפים

פעם בתקופה, הרגולטור בוחר באקראי (לפי תוצאה של הגרלה ציבורית שאפשר לבדוק) כמה משתתפים. משתתף שנבחר נדרש לפתוח את ההתחייבויות שלו, כלומר לחשוף לרגולטור שלו בלבד את הערכים ואת r, ולהראות שהם תואמים את הדיווח הרגולטורי הרגיל שלו (באירופה, דוח Annex IV לפי דירקטיבת AIFMD, ובארה"ב Form PF). פער מהותי מוביל לבירור. הרגולטור כבר רואה את הדוח הרגיל ממילא, ולכן הביקורת לא חושפת מידע חדש. היא רק מוודאת שהדיווח המוצפן תואם לו.

מה נשאר פתוח

הביקורת המדגמית מרתיעה, אבל לא מונעת. משתתף שמשקר גם בדוח הרגיל שלו לא ייתפס כאן. המערכת לא יכולה להיות אמינה יותר מהדיווח הרגולטורי שמתחתיה, וכך כתוב גם בדוחות שהיא מפיקה.

טכנולוגיות ותשתיות תעשייה משלימות

- **Trillian:** מימוש פתוח של Google ליומנים מבוססי Merkle, שעליו נבנו יומני Certificate Transparency. התרומה: יומן מוכח, עם הוכחות הכללה ועקביות מוכנות.
- **drand:** רשת ציבורית של "אקראיות מאומתת" שמפעילים ארגונים כמו Cloudflare ו-Protocol Labs, ומפרסמת מספר אקראי חדש כל כמה שניות. התרומה: בחירת המשתתפים לביקורת לפי מספר שאף אחד, כולל הרגולטור, לא יכול היה לבחור מראש.
- **Form PF-ו AIFMD Annex IV:** הדיווחים הרגולטוריים הקיימים של קרנות באירופה ובארה"ב. התרומה: עוגן אמת חיצוני לבדיקה.

מבנה נתונים

```
LogEntry { index: int; kind: Literal["commitment", "batch", "query", "catalog",
"template", "audit"]; payload: dict; leaf_hash: bytes[32]; ts: datetime }
SignedTreeHead { tree_size: int; root_hash: bytes[32]; timestamp: datetime; signatures:
list[bytes] }   # one per regulator
AuditSelection { period_id; drand_round: int; drand_value: bytes; selected_report_ids:
list[str] }
```

קלט ופלט

קלט: אירועים מכל הפיצ'רים. פלט: יומן ציבורי עם שורש יומי חתום, הוכחות הכללה ועקביות, ורשימת ביקורת מדגמית.

ספריות ופונקציות מרכזיות

שימוש	ספרייה	פעולה
QueueLeaf, GetInclusionProof, GetConsistencyProof	Trillian	יומן
GET /public/{round}	drand	אקראיות ציבורית
חתימה של כל רגולטור על השורש	ed25519-dalek	חתימת שורש

פיצ'ר 10: דוחות לרגולטור ופרסום לציבור (Regulator Reports & Public Release)

מה הפיצ'ר עושה?

התוצר של כל המערכת: לוח לרגולטורים, עם התמונה המצרפית המדויקת והמאומתת, ופרסום תקופתי לציבור עם רעש. וגם ייצוא בפורמט שמודלי לחץ ובדיקות יציבות פיננסית יכולים לקלוט.

איך הפיצ'ר עובד?

הלוח לרגולטור

- **מפת חום:** מדרגות ריבית מול ענפים, בצבע לפי סך החשיפה. תאים חסומים (פחות מ-5 משתתפים) מסומנים בבירור.
- **התפלגות המנוף:** היסטוגרמה של חמש המדרגות.
- **שינוי מתקופה לתקופה:** לכל תא, כמה השתנה, ובאיזה כיוון.
- **תג אימות:** לכל מספר, סימון שהוא עבר את בדיקת ההתחייבויות (פיצ'ר 6), עם קישור לרשומה ביומן.

הפרסום הציבורי

פעם ברבעון, דוח קצר עם תשובות לשאלות הציבוריות בקטלוג, עם רעש, ולצד כל מספר מרווח טעות. מכיוון שהתפלגות הרעש ידועה (Discrete Gaussian עם שונות ידועה), אפשר לחשב מרווח של 95% במדויק ולהציג אותו, כך שקורא יודע כמה המספר מדויק. הדוח כולל גם הצהרה ברורה על מה שהמערכת יודעת ומה לא: מספר המשתתפים, החלק מהשוק שהם מייצגים, ושהנתונים אמינים כמו הדיווח הרגולטורי שמתחתיהם.

ייצוא להתפלגויות

מודלים של לחץ פיננסי צריכים בדיוק את מה שהמערכת יודעת: לא נתון של קרן בודדת, אלא איך המנוף והחשיפות מתפלגים בשוק. הייצוא מפיק את ההיסטוגרמות המצרפיות בפורמט סטנדרטי (JSON לפי סכמה מתועדת, ו-Parquet), שמודל חיצוני יכול לטעון כהתפלגות במקום להמציא אותה.

הרשאות

כל רגולטור רואה בלוח את מה שהקטלוג מאפשר. ההרשאות מנוהלות בשרת זהות ארגוני (OIDC), וכל צפייה בלוח נרשמת ביומן.

טכנולוגיות ותשתיות תעשייה משלימות

- **Grafana או Apache Superset:** כלי לוחות מוכרים, שהרגולטורים כנראה כבר מכירים. התרומה: לא בונים ממשק לוחות מאפס.
- **SDMX:** התקן הבינלאומי להחלפת נתונים סטטיסטיים, שבו משתמשים IMF, ה-ECB ו-BIS. התרומה: הפרסום הציבורי זמין באותו פורמט שבו כלכלנים כבר מורידים נתונים של בנקים מרכזיים.
- **Keycloak (OIDC):** ניהול זהויות והרשאות לרגולטורים.

מבנה נתונים

```
Release {
  release_id: str
  period_id: str
  audience: Literal["regulators", "public"]
  catalog_version: int
  values: dict[str, float]          # query_id -> value
  ci95: dict[str, tuple[float, float]] | None    # public only
  suppressed: list[str]
  verification_ref: int             # LogEntry index
  participants: int
}
DistributionExport { period_id; leverage_hist: list[float]; exposure_by_rate: list[float];
schema_version: str }
```

קלט ופלט

קלט: תשובות מאומתות מפיצ'רים 6 ו-7. פלט: לוח לרגולטורים, דוח ציבורי רבעוני עם מרווחי טעות, וקובצי התפלגות לייצוא.

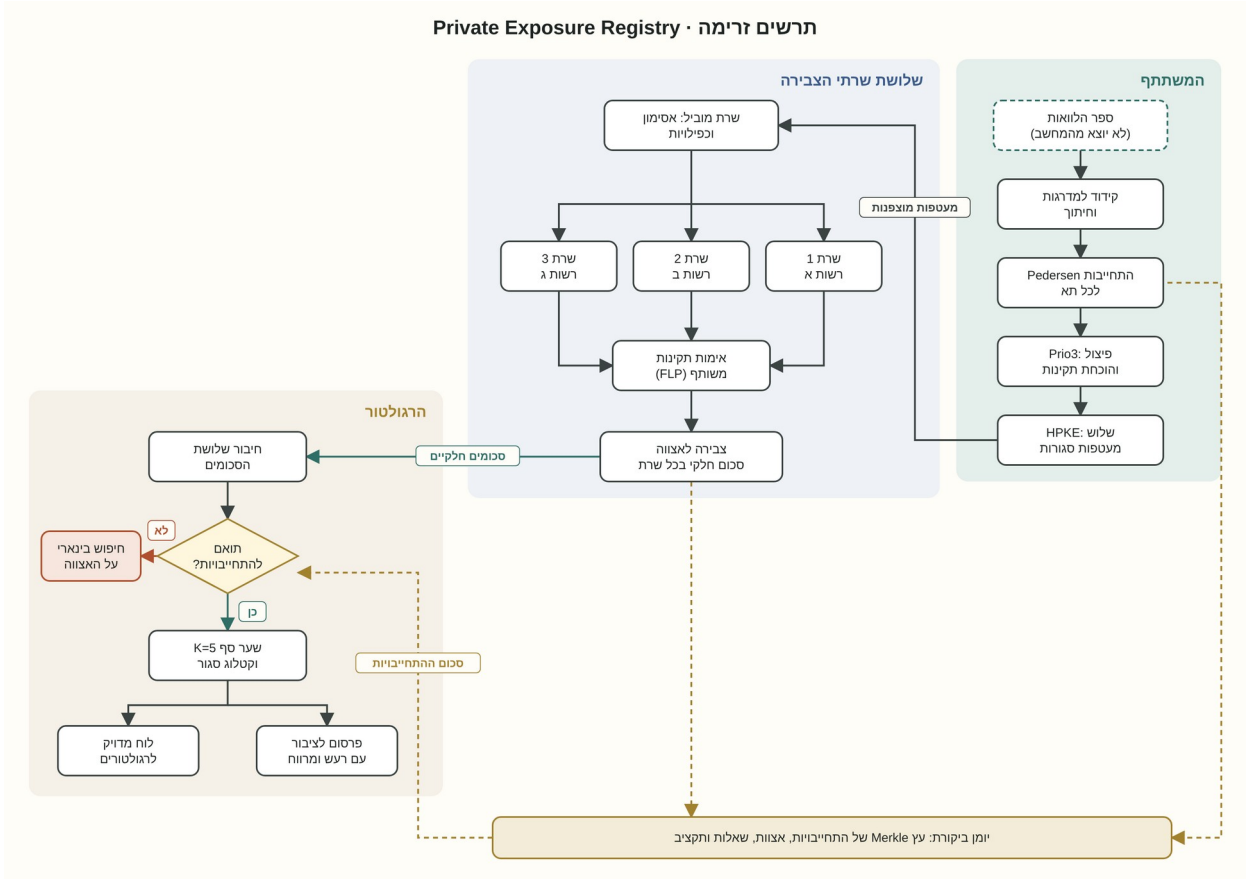
ספריות ופונקציות מרכזיות

פעולה	ספרייה	שימוש
לוחות	Grafana / Superset	חיבור ל-PostgreSQL של הרגולטור
פורמט פרסום	sdmx1 (Python)	בניית הודעות SDMX
הרשאות	Keycloak	OIDC, roles

טבלת טכנולוגיות

פיצ'ר	טכנולוגיות ושפות תכנות	משאבים ושירותים חיצוניים
1. רישום וזהות	Privacy Pass, RSA Blind Signatures, step-ca	GLEIF API
2. קידוד מקומי	Rust, WebAssembly, Apache Arrow	אין (רץ אצל המשתתף)
3. התחייבויות	curve25519-dalek, Ristretto255, ed25519-dalek	אין
4. פיצול והוכחה	libprio-rs (Prio3, VDAF)	אין
5. צבירה	Janus, PostgreSQL, Kubernetes	שלושה שרתים בשלוש רשויות
6. אימות נכונות	curve25519-dalek	אין
7. קטלוג ופרטיות	OpenDP, Python	אין
8. תקשורת ותפעול	hpke-rs, rustls, Envoy, Prometheus	Cloud KMS או HSM
9. יומן ביקורת	Trillian, ed25519-dalek	drand
10. דוחות	Grafana / Superset, sdmx1, Keycloak	אין

תרשים זרימה



התרשים מתאר שלושה עולמות שלא סומכים זה על זה. אצל המשתתף (מימין), הנתונים הגולמיים לא יוצאים: הם מקודדים, ננעלים בהתחייבות, ומפוצלים לשלוש מעטפות מוצפנות, כל אחת לשרת אחר. אצל השרתים (באמצע), כל שרת פותח רק את המעטפה שלו, השרתים בודקים יחד שהדיווח תקין, וצוברים. אצל הרגולטור (משמאל), הסכומים החלקיים מתחברים, נבדקים מול סכום ההתחייבויות, ורק אז עוברים בשער הסף ובקטלוג. משם מתפצלים שני מסלולים: לוח מדויק לרגולטורים, ופרסום עם רעש לציבור. יומן הביקורת (למטה) מקבל אירוע מכל שלב.

מבנה בסיס הנתונים

לכל צד מסד נתונים משלו, ואין מסד נתונים אחד שמחזיק "הכול". כל שרת צבירה מחזיק PostgreSQL משלו עם החלקים שלו בלבד. הרגולטור מחזיק PostgreSQL עם המשתתפים, הקטלוג, התקציב והפרסומים. יומן הביקורת הוא Trillian, שמשתמש ב-MySQL או ב-PostgreSQL כאחסון פנימי. המשתתף לא מחזיק שום מסד נתונים של המערכת.

בכל שרת צבירה: טבלת tasks

- **task_id:** מסוג BYTEA(32), מפתח ראשי.
- **period_id:** מסוג VARCHAR, חובה.
- **template_id:** מסוג VARCHAR, חובה.
- **vdaf_config:** מסוג JSONB, חובה.
- **aggregator_index:** מסוג SMALLINT, חובה (0 מוביל, 1 ו-2 עוזרים).

טבלת reports

- **report_id:** מסוג BYTEA(16), מפתח ראשי.
- **task_id:** מפתח זר ל-tasks.
- **encrypted_share:** מסוג BYTEA, חובה (מוצפן ב-HPKE לשרת הזה).
- **received_at:** מסוג TIMESTAMPTZ, חובה.
- **status:** מסוג ENUM('pending','prepared','aggregated','rejected'), חובה.
- **reject_reason:** מסוג VARCHAR, יכול להיות NULL.
- **batch_id:** מסוג VARCHAR, יכול להיות NULL עד צבירה.
- **expires_at:** מסוג TIMESTAMPTZ, חובה (סוף תקופת השימור).

טבלת used_tokens (במוביל בלבד)

- **token_hash:** מסוג BYTEA(32), מפתח ראשי.
- **period_id:** מסוג VARCHAR, חובה.
- **used_at:** מסוג TIMESTAMPTZ, חובה.

טבלת aggregate_shares

- **task_id:** מפתח זר ל-tasks.
- **batch_id:** מסוג VARCHAR, חובה.
- **share:** מסוג BYTEA, חובה.
- **r_share:** מסוג BYTEA, חובה.
- **report_count:** מסוג INTEGER, חובה.
- **checksum:** מסוג BYTEA(32), חובה.
- **מפתח ראשי משולב:** (task_id, batch_id).

טבלת pending_forwards (במוביל בלבד)

- **report_id:** מפתח זר ל-reports.
- **helper_index:** מסוג SMALLINT, חובה.
- **encrypted_share:** מסוג BYTEA, חובה.
- **queued_at:** מסוג TIMESTAMPTZ, חובה.
- **attempts:** מסוג INTEGER, ברירת מחדל 0.

אצל הרגולטור: טבלת participants

- **lei:** מסוג CHAR(20), מפתח ראשי.
- **regulator_id:** מסוג VARCHAR, חובה.
- **sector:** מסוג VARCHAR, חובה.
- **verified_at:** מסוג TIMESTAMPTZ, חובה.
- **status:** מסוג ENUM('active','suspended'), חובה.

טבלת token_issuance

- **lei:** מפתח זר ל-participants.
- **period_id:** מסוג VARCHAR, חובה.

- **issued_at:** מסוג TIMESTAMPTZ, חובה.
- **מפתח ראשי משולב:** (lei, period_id). מבטיח אסימון אחד לגוף לתקופה. הטבלה לא מכילה את האסימון עצמו, ולכן לא ניתן לקשר אותו לדיווח.

טבלת templates ו-catalog_versions

- **templates:** template_id (מפתח ראשי), json_schema (JSONB), clip_musd, bits_per_cell, approved_at.
- **catalog_versions:** version (מערך של מזהי queries (JSONB), approved_by (מפתח ראשי), differencing_check_passed (BOOLEAN, חייב להיות true), רגולטורים, חייב להכיל שלושה).

טבלת batches

- **batch_id:** מסוג VARCHAR, מפתח ראשי.
- **period_id:** מסוג VARCHAR, חובה.
- **n_reports:** מסוג INTEGER, חובה.
- **n_rejected:** מסוג INTEGER, חובה.
- **verified:** מסוג BOOLEAN, חובה.
- **verification_log_index:** מסוג BIGINT, חובה (הפניה ליומן).

טבלת budget_ledger

- **entry_id:** מסוג SERIAL, מפתח ראשי.
- **period_id:** מסוג VARCHAR, חובה.
- **query_id:** מסוג VARCHAR, חובה.
- **rho:** מסוג DOUBLE PRECISION, חובה, חיובי.
- **spent_at:** מסוג TIMESTAMPTZ, חובה.
- **אילוץ:** סכום rho לכל period_id לא עולה על התקציב שהוגדר (נבדק בטריגר לפני הכנסה).

טבלת releases

- **release_id:** מסוג UUID, מפתח ראשי.
- **period_id, audience (ENUM:** regulators, public), catalog_version, values (JSONB), ci95 (JSONB, יכול להיות NULL), suppressed (מערך), participants (INTEGER), verification_log_index (BIGINT).

טבלת audit_selections

- **period_id:** מפתח ראשי.
- **drand_round:** מסוג BIGINT, חובה.
- **drand_value:** מסוג BYTEA, חובה.
- **selected_report_ids:** מערך של BYTEA(16), חובה.
- **outcome:** מסוג JSONB (תוצאות הבירור לכל נבחר), יכול להיות NULL עד סיום.

מה האפיון סגר, ומה נשאר פתוח

- **נסגר:** חמישה מתוך ששת האיומים ממודל האיום קיבלו מענה מלא: סיביל (LEI ואסימון אחד לגוף), שרת שמשבש (בדיקה הומומורפית מול התחייבויות), הכחשה (התחייבויות וקבלות), הפרשים (קטלוג סגור עם

בדיקה אוטומטית ותקציב), ושרת שנופל (החלטה מנומקת ותפעול). והכי חשוב: הליבה נשענת על סטנדרט בפיתוח של IETF ועל קוד שכבר רץ בייצור, לא על קריפטוגרפיה שהמצאנו.

- **נשאר פתוח:** האיום השישי, משתתף שמשקר. הוכחות התקינות מוכיחות טווח, לא אמת. הביקורת המדגמית מרתיעה, ומחזקת את הקשר לדיווח הרגולטורי הקיים, אבל המערכת לא אמינה יותר ממנו.
- **נשאר פתוח:** המערכת יודעת מה מצב השוק עכשיו, בדיוק ובלי לחשוף אף אחד. מה שהיא לא יודעת לעשות לבד הוא לענות על השאלה "ומה יקרה אם הריבית תעלה עוד נקודה?". היא מראה את התמונה, לא את מה שיקרה לה.